Ski Lift Line Accurate Wait Time

Talon Finehout

Andrew Nere

CS - 420

This project aims to provide an accurate ski lift line wait time so that resorts can inform skiers about how long it will take to get onto the lift and how many people are currently in line. This helps skiers better plan their runs, allowing them to spend less time waiting and more time enjoying the mountain. By using Ultralytics' pretrained models for optimal accuracy and detection, I was able to create a YOLO11 model that performs real-time object detection and tracking.

YOLO11 is the latest iteration in the Ultralytics YOLO series, known for its excellent accuracy, speed, and efficiency. I chose this model because of its versatility, especially for tracking skiers in crowded and visually complex scenes. Some advantages of YOLO11 include enhanced feature extraction for better detection in dense environments and 22% fewer parameters than YOLOv8, making it faster to compile and more efficient, especially important since I am currently working with a CPU. In addition to its efficiency, YOLO11 supports various tasks such as object detection, instance segmentation, pose and keypoint estimation, oriented detection, and classification. This versatility enables more complex computer vision tasks, such as tracking both the positions and poses of people and analyzing how long they remain in specific areas.

YOLO11 comes in several model variants that differ in size, parameter count, accuracy, and speed. I opted for YOLO11n, which offers an average precision (mAP) of 39.5 but runs the fastest on a CPU. Following instructions from the Ultralytics GitHub, I downloaded the YOLO models and trained mine on the built-in COCO8 dataset, which contains only 8 images (4 for training and 4 for validation). While small, this dataset is tailored for object detection and enables quick training. It can detect up to 80 different classes, including the "person" class, which was the focus of my validation.

After training for 100 epochs and validating for 16 epochs using only the "person" class, the model achieved a recall of 55% and a low precision of 10%. While these values may seem underwhelming, the model still performs well in practice, detecting people in real-world images with around 60% confidence, even when they appear small or partially obscured. Based on these results, I proceeded to build the rest of my computer vision project using OpenCV and NumPy.

This project builds on the basic functionality explored in previous assignments, such as loading a saved model and applying it to a video or image for prediction. In this case, the YOLO11 model includes a track() function, which not only detects objects using YOLO's core capabilities but also follows those objects across video frames. For this project, the model is limited to detecting only the "person" class.

While YOLO11 can generate bounding boxes that include the detected class and confidence level, I chose to implement custom bounding boxes that instead display how long each person has been visible in the frame. This helps visualize how long a skier has been waiting in line. By tracking the duration that each detected ID remains within the user-defined Region of Interest (ROI), the system can estimate an average wait time by summing the time each person spends in line and dividing it by the total number of people tracked.

One of the key challenges in making this an accurate wait-time detection model is determining the actual duration a person has been in the frame. This uncertainty arises because I do not know the exact frame rate of the ski lift cameras being used. I've assumed a standard 30 frames per second (FPS), which is common for most cameras. However, as Roboflow's blog *"Monitor and Analyze Retail Queues Using Computer Vision"* explains, live-streamed video can be inconsistent due to lag or hardware limitations. Additionally, the time it takes for my model to

process each frame introduces further delay, which can distort both frame rate and timing accuracy.

Despite these limitations, the project successfully provides the basic functionality needed to estimate ski lift wait times and count the number of people currently in line. With interactive controls that allow users to adjust the ROI, the system is adaptable to any setting where video footage of a queue is available. If I were to continue this project, I would aim to train the model on a GPU to enhance accuracy, improve processing speed, and deliver more precise wait time estimates overall.

Time Sheet

This was a solo project, these are estimated times since I was going in between all at once.

- Recording/editing data > 5 hours
- Researching YOLO model ~ 2 hours
- Getting video to work with the model ~ 2 hours (This was a hard time)
- Design model ~ 2 hours
- Train final and test ~ 4 hours (I kept messing with opencv file until I was happy)
- Final conclusion ~ 4 hours

Sources

"Monitor and Analyze Retail Queues Using Computer Vision." *Roboflow Blog*, 15 Apr. 2024, blog.roboflow.com/monitor-retail-queues/. Accessed 09 May 2025.

Ultralytics. "COCO8." *Ultralytics YOLO Docs*, 18 Apr. 2025, docs.ultralytics.com/datasets/detect/coco8/. Accessed 09 May 2025.

Ultralytics. "How Can I Change Number of Classes of Yolov11 Model to Two? · Issue #18525 · Ultralytics/Ultralytics." *GitHub*, github.com/ultralytics/ultralytics/issues/18525. Accessed 09 May 2025.

Ultralytics. "How to Save and Then Reload a Custom Finetuned Yolo Model (with Custom Classes) · Issue #10297 · Ultralytics/Ultralytics." *GitHub*, github.com/ultralytics/ultralytics/issues/10297. Accessed 09 May 2025.

Ultralytics. "Hyperparameter Tuning." *Ultralytics YOLO Docs*, 6 Apr. 2025, docs.ultralytics.com/guides/hyperparameter-tuning/. Accessed 09 May 2025.

Ultralytics. "Track." *Ultralytics YOLO Docs*, 30 Apr. 2025, docs.ultralytics.com/modes/track/#features-at-a-glance. Accessed 09 May 2025.

Ultralytics. "Train." *Ultralytics YOLO Docs*, 3 May 2025, docs.ultralytics.com/modes/train/. Accessed 09 May 2025.

Ultralytics. "Ultralytics YOLO11." *Ultralytics YOLO Docs*, 26 Feb. 2025, docs.ultralytics.com/models/yolo11/. Accessed 09 May 2025.

Ultralytics. “Ultralytics/Ultralytics: Ultralytics Yolo11.” *GitHub*, github.com/ultralytics/ultralytics?tab=readme-ov-file. Accessed 09 May 2025.

Ultralytics. “Val.” *Ultralytics YOLO Docs*, 20 Mar. 2025, docs.ultralytics.com/modes/val/#arguments-for-yolo-model-validation. Accessed 09 May 2025.